



نام درس: سیستم های توزیع شده
نویسنده: محمد امانلو - ۸۱۰۱۰۰۰۸۴



تمرین دستی
شماره ۱

شماره سوال ۱

دو تکه کد به زبان Go بنویسید که دو goroutine متفاوت آن‌ها را اجرا می‌کنند و در نتیجه انحصار متقابل اشتباه دچار بن‌بست شده باشند

بن‌بست (deadlock) وضعیتی است که در آن دو یا چند goroutine هر کدام قفلی را در اختیار دارند و منتظر آزاد شدن قفل دیگری هستند در نتیجه هیچ‌یک پیشرفت نمی‌کنند و برنامه متوقف می‌شود کانال بدون بافر (unbuffered channel) کانالی است که هیچ فضای میانی برای نگهداری داده ندارد وقتی یک goroutine مقداری به این کانال می‌فرستد تا زمانی که goroutine دیگری آن مقدار را دریافت نکند ارسال بلوکه می‌شود و بالعکس اگر دریافت درخواست شود اما مقداری ارسال نشده باشد دریافت بلوکه می‌شود

مثال اول : دو قفل (Mutex) با ترتیب معکوس:

در این مثال دو قفل مجزا به نام‌های muA و muB تعریف کرده‌ایم دو goroutine هم‌زمان اجرا می‌شوند goroutine نخست ابتدا قفل muA را می‌گیرد و سپس پس از مکث کوتاهی دنبال قفل muB می‌گردد goroutine دوم ابتدا قفل muB را می‌گیرد و بعد تلاش می‌کند قفل muA را به دست آورد هر goroutine قفل اولیه را در اختیار دارد و منتظر آزاد شدن قفل دوم است بنابراین هر دو در حالت بن‌بست می‌مانند و برنامه دیگر جلو نمی‌رود

```
1 var muA sync.Mutex
2 var muB sync.Mutex
3 var wg sync.WaitGroup
4
5 func main() {
6     wg.Add(2)
7     go func() {
8         defer wg.Done()
```

```

9      fmt.Println("goroutine_1:locked_muA")
10     muA.Lock()
11     time.Sleep(300 * time.Millisecond)
12     fmt.Println("goroutine_1:trying_to_lock_muB")
13     muB.Lock()
14     muB.Unlock()
15     muA.Unlock()
16 }()
17 go func() {
18     defer wg.Done()
19     fmt.Println("goroutine_2:locked_muB")
20     muB.Lock()
21     time.Sleep(300 * time.Millisecond)
22     fmt.Println("goroutine_2:trying_to_lock_muA")
23     muA.Lock()
24     muA.Unlock()
25     muB.Unlock()
26 }()
27 wg.Wait()
28 fmt.Println("this_line_never_prints;_deadlock_occurred")
29 }

```

اگر این فایل را اجرا کنیم ما در خروجی ابتدا می بینیم

۱. «goroutine 1: locked muA» که نشان می دهد goroutine نخست عملیات Lock روی muA را انجام داده است

۲. پس از مکث کوتاه «goroutine 2: locked muB» که یعنی goroutine دوم قفل muB را به دست آورده است

۳. سپس goroutine اول تلاش می کند قفل muB را بگیرد اما چون muB در اختیار goroutine دوم است goroutine اول بلوکه می شود

۴. goroutine دوم نیز پس از تاخیر خودش به سراغ قفل muA می رود اما muA در اختیار goroutine اول قرار دارد بنابراین او هم بلوکه می شود

۵. در نتیجه هر دو goroutine برای همیشه منتظر آزاد شدن قفل دوم باقی می ماند و دستور wg.Wait() هیچ گاه خاتمه پیدا نمی کند جمله پایانی نیز نمایش داده نمی شود

مثال دوم : دو کانال بدون بافر:

در این مثال دو کانال بدون بافر (unbuffered) با نام های ch1 و ch2 ایجاد کرده ایم دو goroutine موازی اجرا می شوند goroutine اول عدد 1 را به ch1 می فرستد و سپس منتظر دریافت از ch2 می شود goroutine دوم عدد 2 را به ch2 می فرستد و سپس منتظر دریافت از ch1 می شود از آنجا که هر دو کانال بدون بافر هستند عملیات ارسال (send) در هر کانال بلوکه می شود تا طرف مقابل (receive) عمل کند در نتیجه هر دو goroutine در وضعیت بن بست قرار می گیرند و برنامه قفل می شود

```

1 func main() {
2     ch1 := make(chan int)
3     ch2 := make(chan int)
4     go func() {
5         fmt.Println("goroutine_1: send 1 to ch1")
6         ch1 <- 1
7         fmt.Println("goroutine_1: waiting to receive from ch2")
8         <-ch2
9         fmt.Println("goroutine_1: if this prints ,no deadlock")
10    }()
11    go func() {
12        fmt.Println("goroutine_2: send 2 to ch2")
13        ch2 <- 2
14        fmt.Println("goroutine_2: waiting to receive from ch1")
15        <-ch1
16        fmt.Println("goroutine_2: if this prints ,no deadlock")
17    }()
18    select {}
19 }

```

اگر این فایل را اجرا کنیم ما در خروجی خواهیم دید

۱. «goroutine 1: send 1 to ch1» و «goroutine 2: send 2 to ch2» تقریباً همزمان چاپ می شوند

۲. goroutine اول پس از ارسال به ch1 بلوکه می شود تا goroutine دوم داده را از ch1 دریافت کند اما goroutine دوم هنوز در مرحله ارسال به ch2 بلوکه است

۳. goroutine دوم پس از ارسال به ch2 بلوکه می شود تا goroutine اول داده را از ch2 دریافت کند در حالی که goroutine اول هنوز در ارسال به ch1 بلوکه مانده است

۴. بنابراین هر دو goroutine در وضعیت بلوکه باقی می ماند و هیچ یک از چاپ های «if this prints, no» را مشاهده نمی کنیم

۵. همچنین دستور select {} هیچ گاه خاتمه نمی یابد و برنامه تا ابد در حالت بن بست باقی می ماند

در هر دو مثال وقتی فایل ها را اجرا کنیم می بینیم که پس از چند خط خروجی اولیه بن بست رخ داده و برنامه دیگر پیشرفتی ندارد

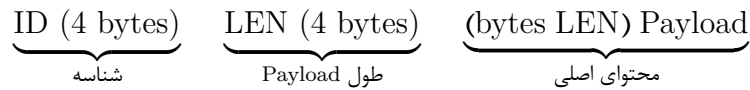
شماره سوال ۲

می دانیم که می توانیم روی یک اتصال TCP یک دنباله از بایت ها را ارسال کنیم که تعداد آن ها دلخواه است. فرض کنید بخواهیم بر روی یک اتصال TCP چندین درخواست و پاسخ را ارسال کنیم. چگونه می توان چنین پروتکلی را طراحی کنیم؟

وقتی می خواهیم روی یک اتصال TCP چندین درخواست و پاسخ را پشت سرهم بفرستیم، باید اول بدانیم که TCP خودش فقط یک جریان بایت (byte stream) ارائه می کند و هیچ مرزی بین پیام های مختلف قائل نیست. یعنی اگر بدون سازوکار خاصی شروع کنیم به فرستادن چند پیام پی درپی، سمت گیرنده مقابل نمی فهمد پیام اول کجا تمام شده و پیام بعدی از کجا شروع شده. برای رفع این مشکل، باید خودمان کاری کنیم تا هر پیام یک frame داشته باشد؛ یک روش مرسوم این است که قبل از هر پیام، یک length-prefix بفرستیم. فرض کنید می خواهیم یک payload دلخواه (مثلا یک رشته JSON یا داده باینری) را ارسال کنیم. ابتدا طول آن payload را به صورت یک عدد ثابت طول (مثلا ۴ بایت با ترتیب بایت Big-Endian یا Little-Endian) می فرستیم و بلافاصله بعدش همان تعداد بایت پیام را می فرستیم. با این روش گیرنده اولین چهار بایت را می خواند، این عدد را تفسیر می کند و می داند باید چند بایت بعدی را بخواند تا پیام کامل شود. وقتی آن N بایت را کامل گرفت، دوباره می چرخد سراغ چهار بایت بعدی به عنوان طول پیام دوم و الی آخر.

در عمل معمولا برای هر پیام علاوه بر آن فیلد طول، یک فیلد شناسه هم می گذارند (مثلا عدد ۴ بایتی به Big-Endian یا Little-Endian می نویسند) تا اگر چند درخواست هم زمان داشته باشیم و پاسخ ها به ترتیب دلخواه برسند، بتوانیم

هر پاسخ را به درخواست مربوطش وصل کنیم. در این حالت ساختار پیام می شود چیزی مثل:



گیرنده اول هشت بایت را می خواند، فرض می کند عدد اول (مثلا ۱۷) شناسه پیام است و عدد دوم (مثلا ۱۲۳) طول payload را نشان می دهد. سپس باید ۱۲۳ بایت بعدی را بخواند تا Payload را کامل دریافت کند. وقتی آن ۱۲۳ بایت را خواند، یک پیام کامل دارد که می تواند بر اساس ID یا نوع پیام آن را پردازش کند. همین طور اگر بخواهیم ده ها پیام را پی در پی ارسال کنیم، کافی است همین الگو را برای هر پیام تکرار کنیم: اول هدر هشت بایتی (ID و LEN) و بعد Payload به تعداد LEN بایت، و سپس هدر بعدی و Payload بعدی و الی آخر.

یک روش دیگر هم استفاده از جداساز (delimiter) است؛ یعنی به جای اینکه حتما طول پیام را قبلش بفرستیم، یک الگوی خاص تعریف می کنیم که نشان دهد «این نقطه انتهای پیام است». مثلا در بسیاری از پروتکل های متنی از دنباله دو بایت `\r\n` به عنوان پایان پیام استفاده می کنند. در این حالت هر پیام را سریال می کنیم (مثلا JSON می شود رشته UTF-8) و پس از آن کاراکترهای `\r\n` می فرستیم. گیرنده یک بافر درونی نگه می دارد و هر بار تعدادی بایت از TCP می خواند و به انتهای بافر اضافه می کند. وقتی در بافر دنباله `\r\n` پیدا شد، می فهمد که تا آن نقطه یک پیام کامل دریافت کرده است. سپس آن قدر بایت قبل از `\r\n` را جدا می کند و برای پردازش تحویل می دهد و باقی بافر را (پس از آن `\r\n`) نگه می دارد برای پیام های بعدی. در این روش باید مراقب باشیم خود Payload حاوی همان دنباله جداکننده نباشد؛ در غیر این صورت قبل از اینکه پیام به پایان رسیده، اشتباها گیرنده فکر می کند پایان پیام است. برای همین اگر احتمال دارد در payload از همان الگو استفاده شود، معمولا باید payload را قبل از ارسال encode کنیم یا از دنباله ای استفاده کنیم که مطمئن هستیم در داده اصلی تکرار نمی شود.

در عمل اگر از یک پروتکل متن محور شبیه HTTP/1.1 استفاده کنیم، در ابتدا تعدادی خط هدر داریم که با `\r\n` پایان می یابند و پایان بخش هدرها با یک خط خالی (دو بار `\r\n` پشت سرهم) مشخص می شود. اگر بخواهند بدنه HTTP بفرستند، معمولا از هدر `Content-Length: <number>` استفاده می کنند تا گیرنده بداند باید دقیقا چند بایت را پس از هدرها به عنوان بدنه بخواند. این ترکیب در HTTP، هم از delimiter خطی (برای پایان هدر) و هم از پیش عنوان طول (برای پایان body) استفاده می کند.

یک نکته خیلی مهم دیگر این است که وقتی قصد داریم چند درخواست و پاسخ را روی یک کانکشن TCP جابه جا کنیم، معمولا حالت همزمانی (concurrency) را هم مدنظر داریم. فرض کنید چند goroutine در کلاینت داریم که می خواهند درخواست بفرستند. بهتر است یک goroutine واحد مسئول نوشتن روی سوکت باشد و همه goroutine های دیگر وقتی می خواهند درخواست بفرستند، پیامشان را در یک کانال داخلی (Go channel) قرار دهند. آن writer goroutine مرتب از آن کانال می خواند و Write می کند روی سوکت، تا مطمئن باشیم دو goroutine همزمان وارد Write نشوند و تداخل پیش نیاید. از طرف دیگر، برای خواندن هم یک goroutine واحد داریم که دائم Read می کند؛

داده ها را در بافر داخلی می چینند، تا وقتی که یک پیام کامل نرسیده صبر می کند، و وقتی پیام کامل شد، آن را تحویل یک کانال دیگر یا تابع پردازش اصلی می دهد. با این ساختار ساده، هیچ وقت چند goroutine به طور همزمان روی سوکت نمی خوانند یا نمی نویسند و احتمال تداخل یا بلوک شدن ناخواسته کاهش می یابد.

اگر قرار باشد سرور همزمان چندین درخواست را پاسخ بدهد و ترتیب پاسخ ها الزاما مثل ترتیب درخواست ها نباشد (مثلا درخواستی ممکن است پردازش طول بکشد و بعد پاسخش دیر برسد)، حتما باید از شناسه (ID) استفاده کنیم. یعنی وقتی هر goroutine درخواستش را می سازد، یک ID یکتا انتخاب می کند و داخل هدر پیام (مثلا همزمان با طول) می فرستد. سمت سرور که پاسخ را آماده می کند، همان ID را در پاسخ می گذارد. هنگام دریافت جواب، گیرنده می تواند بر اساس ID بفهمد این پاسخ مربوط به کدام درخواست بوده و آن را به goroutine درست تحویل بدهد. اگر ترتیب FIFO برایمان کافی باشد (یعنی مطمئنیم ترتیب پاسخ دقیقا مثل ترتیب درخواست خواهد بود)، ممکن است نیازی به ID نداشته باشیم، اما اغلب مواقع که چند درخواست همزمان داریم و پاسخ ها ممکن است به ترتیب دلخواه برسند، ID ضروری است تا ارتباط برقرار باشد.

شماره سوال ۳

کاربرد یک زبان تعریف رابط IDL چیست؟ قطعه کد زیر که به زبان Protocol Buffers نوشته شده است را توضیح دهید.

```
message PaymentRequest {
  message Card {
    required string cardNumber = 1;
    optional int32 expiryMonth = 2;
    optional int32 expiryYear = 3;
    optional int32 CVC = 4;
  }
  enum Currency { GBP = 1; USD = 2; }
  required Card card = 1;
  required int64 amount = 2;
  required Currency currency = 3;
}

message PaymentStatus {
  required bool success = 1;
  optional string errorMessage = 2;
}

service PaymentService {
  rpc ProcessPayment(PaymentRequest) returns (PaymentStatus) {}
}
```

زبان تعریف رابط IDL برای تعریف interfaces بین اجزای نرم افزاری به خصوص در سیستم های توزیع شده به کار می رود. به وسیله IDL می توان ساختار داده (data structures) و services را مستقل از زبان برنامه نویسی مشخص کرد. در

نتیجه ارتباط بین اجزای مختلف در سیستم آسان می شود. هدف این است که سیستم هایی که ممکن است با زبان های مختلف نوشته شده باشند بتوانند به راحتی با یکدیگر ارتباط برقرار کنند. در واقع IDL سربار فرایندهای serialization و deserialization را از برنامه نویسی می گیرد؛ بدون انجام خودکار این کارها زمان development افزایش می یابد و احتمال خطا بالا می رود. همچنین با هر تغییر کوچک در data schemas باید تمام توابع serialize و deserialize تغییر کنند، ولی با IDL برنامه نویسی فقط نیاز دارد با همین data structures در زبان خود کار کند، بدون نگرانی در جزئیات تبدیل داده ها به بایت و برعکس.

در Protocol Buffers (protobuf)، زیرمجموعه ای از IDL ارائه می شود که با فایل های proto. تعریف می شود. کامپایلر protoc از این فایل ها code تولید می کند تا کلاینت و سرور در زبان های مختلف (مثلاً Go، Java، Python) طبق قرارداد مشترک عمل کنند.

۱. تعریف message ها:

این کد دو message تعریف کرده که مانند struct در زبان های برنامه نویسی عمل می کنند. هر message برای تعریف data type جدید استفاده می شود. در هر message می توانیم required یا optional بودن فیلدها را مشخص کنیم. عدد بعد از علامت مساوی شماره field است. شماره field یک عدد صحیح مثبت است که در فرایند serialization پیام به binary، برای شناسایی فیلدهای مختلف به کار می رود و امکان forward و backward compatibility را فراهم می کند.

۱.۱. **message PaymentRequest** این message ساختار PaymentRequest را تعریف می کند که برای ارسال اطلاعات پرداخت از کلاینت به سرور استفاده می شود.

message Card

required string cardNumber = 1; این فیلد شماره کارت را به شکل یک رشته UTF-8 نگه می دارد. در protobuf وقتی required مشخص است، ارسال این فیلد اجباری است و اگر در پیام نباشد در زمان (de)serialization خطا رخ می دهد.

optional int32 expiryMonth = 2; تاریخ انقضای کارت به شکل عدد صحیح ۳۲ بیتی که ماه را مشخص می کند (از ۱ تا ۱۲). به دلیل optional بودن، در صورت نبودن این فیلد، مقدار پیش فرض صفر (۰) استفاده می شود؛ اما کلاینت می تواند این فیلد را حذف کند اگر نیازی به آن نداشته باشد.

optional int32 expiryYear = 3; سال انقضای کارت به صورت عدد صحیح ۳۲ بیتی. مشابه expiryMonth، اگر ارسال نشود مقدار پیش فرض صفر خواهد بود.

optional int32 CVC = 4; کد امنیتی (CVC) کارت به صورت عدد صحیح ۳۲ بیتی. اختیاری است و می توان آن را حذف کرد مگر اینکه منطق کسب و کار نیاز به آن داشته باشد.

enum Currency { GBP = 1; USD = 2; } این بخش یک enum به نام Currency تعریف می کند. enum در protobuf فهرستی از مقادیر نام گذاری شده است که هر کدام یک integer را به خود اختصاص می دهند.

required Card card = 1; این فیلد از نوع Card است که همان message Card تعریف شده در بالا است. به این ترتیب ساختار تودرتو داریم. اگر کلاینت این فیلد را ارسال نکند، خطای required field missing رخ می دهد.

required int64 amount = 2; این فیلد مبلغ پرداختی (amount) را به صورت عدد صحیح ۶۴ بیتی نگه می دارد. amount باید مقدار دلخواهی به عنوان مبلغ باشد و ارسال آن اجباری است؛ در صورت نبودن مقدار، پیام اعتبارسنجی نمی گذرد.

required Currency currency = 3; این فیلد نشان می دهد مبلغ بالا به چه ارزی پرداخت می شود؛ از نوع Currency است که در بالا تعریف شد. اگر ارسال این فیلد نباشد، خطا رخ می دهد.

۲.۱. message PaymentStatus این message ساختار PaymentStatus را تعریف می کند که سرور پس از دریافت PaymentRequest و پردازش پرداخت، باز می گرداند:

required bool success = 1; یک مقدار boolean است که نشان می دهد آیا پرداخت موفق بوده (true) یا ناموفق (false). چون required است حتماً باید ارسال شود و اگر در پیام نباشد، خطای required field missing رخ می دهد.

optional string errorMessage = 2; در صورتی که success برابر false باشد، می توان یک string شامل توضیح خطا ارسال کرد. اگر success true باشد، معمولاً این فیلد حذف می شود. در حالت optional، اگر ارسال نشود مقدار پیش فرض رشته خالی در نظر گرفته می شود.

۲. تعریف service:

در بخش service ما یک رابط RPC تعریف می کنیم. service در protobuf معادل گروهی از متدهای RPC است که کلاینت می تواند روی سرور فراخوانی کند.

service PaymentService {} این service ابتدا یک service به نام PaymentService ایجاد می کنیم. این سرویس مجموعه ای از متدهای RPC را تعریف می کند.

rpc ProcessPayment(PaymentRequest) returns (PaymentStatus) {} این خط یک متد RPC به نام ProcessPayment تعریف می کند که از نوع unary RPC است (یک درخواست و یک پاسخ). ساختار این

فراخوانی به این شکل است که کلاینت ابتدا یک شیء `PaymentRequest` را سریالیزه می کند (طبق تعریف فیلدها و شماره field) و روی `TCP` به سرور می فرستد. سرور پیام دریافت شده را دسیرالیزه می کند، منطق کسب و کار پرداخت را اجرا می کند (مثل اعتبارسنجی کارت، برقراری ارتباط با درگاه بانکی و ...) و در نهایت یک شیء `PaymentStatus` تولید می کند. این شیء را سریالیزه کرده و به کلاینت برمی گرداند.

□ در `Protocol Buffers` هر فیلد یک شماره یکتا (field number) دارد که در مشخصه `1 =`، `2 =` و غیره دیده می شود. این عدد در ساختار باینری پیام به صورت tag ذخیره می شود که شامل شماره فیلد و `wire type` است. `wire type` نشان می دهد داده فیلد چگونه کدگذاری می شود.

□ وقتی سرور یا کلاینت پیام را دسیرالیزه می کند، ابتدا tag را مطالعه می کند: field number را استخراج می کند و سپس بسته به نوع داده بایت های بعدی را می خواند.

□ `required` به این معنی است که پیغام باید حتماً آن فیلد را داشته باشد؛ اگر نباشد، در حین دسیرالیزه کردن خطا می دهد. `optional` به این معنی است که اگر فیلد ارسال نشود، مقدار پیش فرض (صفر برای عددها، رشته خالی برای `string` و `false` برای `bool`) در نظر گرفته می شود.

□ `enum` نیز به `protobuf` اجازه می دهد یک مجموعه محدود از مقادیر صحیح را با نام های خوانا تعریف کند. در اینجا `Currency` به جای ارسال یک `int32` آزاد، دو مقدار `GBP` یا `USD` را محدود می کند. این باعث جلوگیری از خطا در کد کلاینت و سرور می شود که فقط مقادیر مجاز را ارسال کنند.

شماره سوال ۴

دو ارتش از انسان ها و اِلِف ها شهر موردور را محاصره کرده اند و قصد حمله به آن را دارند. شما فرماندهی ارتش انسان ها را به عهده دارید. مواضع دفاعی موردور نسبتاً خوب است و اگر یک ارتش به تنهایی به آن حمله کند حتماً شکست می خورد. اما اگر دو ارتش به آن حمله کنند موردور سقوط می کند. تنها راه ارتباطی دو فرمانده ارسال یک سرباز به عنوان پیام رسان است که البته ممکن است در بین راه دستگیر شود و پیام را به مقصد نرساند. برای هماهنگی با فرمانده دیگر چه متنی را پیشنهاد می کنید و استراتژی شما برای غلبه بر موردور چیست؟

۱. عدم قطعیت در هماهنگی، چرا هیچ راه قطعی ای وجود ندارد؟

مساله آنجا آشکار می شود که تنها راه ارتباطی بین دو فرمانده فرستادن یک سرباز (messenger) است و ممکن است در مسیر اسیر شود و پیام به مقصد نرسد. در نتیجه:

□ هر بار که یک پیام ارسال می شود، احتمال p وجود دارد که آن پیام اسیر شود و هیچ تضمینی برای رسیدن ندارد.

□ حتی اگر فرمانده انسان‌ها پیام «حمله در چهارشنبه هفته آینده با طلوع آفتاب» را بفرستد و فرمانده الف‌ها پاسخ تایید بدهد، همان پاسخ تایید هم ممکن است در مسیر گم شود.

□ فرض کنید دو مرحله تایید داریم: اول پیام اصلی، سپس پاسخ الف‌ها و بعد باز تایید نهایی انسان‌ها. اگر «آخرین تایید» اسیر شود، یک طرف می‌پندارد هم‌زمانی رخ داده و طبق برنامه عمل می‌کند، و طرف دیگر معتقد است هنوز هماهنگی کامل نشده و حمله را به تاخیر می‌اندازد؛ در نتیجه حمله هم‌زمان منتفی می‌شود.

□ این ساختار *unreliable communication channel* یعنی هیچ پیامی نمی‌تواند تضمین کند که به مقصد رسیده و دریافت شده‌است. حتی اگر تعداد تاییدها را زیاد کنیم، باز هم احتمال نرسیدن «آخرین بسته» (پیام یا تایید) همیشه بالاست و هرگز به صفر نمی‌رسد.

□ از این رو، اثبات می‌شود که هیچ پروتکلی در چنین شرایطی نمی‌تواند هماهنگی قطعی فراهم کند؛ فقط می‌توان احتمال خطا را به دلخواه کوچک کرد ولی صفر نه.

به همین دلیل تنها راه مطمئن برای هماهنگی، تکیه بر یک مرجع از پیش توافق شده (مثلاً زمان مشخص بر اساس تقویم) است که نیازی به تایید پیام‌رسان لحظه‌ای نداشته باشد.

۲. استراتژی‌های پیشنهادی برای هماهنگی

۱. تعیین زمان از پیش بدون نیاز به تایید: اگر بتوانیم از قبل یک زمان دقیق را توافق کنیم مثلاً «چهارشنبه هفته آینده با طلوع آفتاب حمله کنیم دیگر نیازی به ردوبدل مکرر پیام نیست. در این حالت، حتی اگر پیام اول (اطلاع از زمان) به فرمانده الف‌ها نرسد، هر دو طبق توافق از پیش انجام‌شده عمل می‌کنند. وقتی آن روز فرا برسد، هر دو هم‌زمان هجوم خواهند برد و موردور بدون دفاع هم‌زمان، شکست می‌خورد.

۲. ارسال و دریافت چند مرحله‌ای پیام با تایید دریافت (ACK): اگر بخواهیم احتمال هماهنگی را بیشتر کنیم، می‌توانیم چند مرحله پیام و تایید (ACK) ترتیب دهیم:

۱. فرمانده انسان‌ها پیام «حمله در چهارشنبه هفته آینده با طلوع آفتاب» را می‌فرستد.

۲. اگر فرمانده الف‌ها آن پیام را دریافت کند، با پیام «پیغام شما دریافت شد؛ ما نیز چهارشنبه هفته آینده با طلوع آفتاب حمله می‌کنیم» پاسخ می‌دهد.

۳. اگر این پاسخ نرسد، انسان‌ها دوباره همان پیام را ارسال می‌کنند و منتظر پاسخ تایید می‌مانند.

۴. بعد از اینکه یک بار پاسخ تایید رسید، انسان‌ها یک بار دیگر تایید دریافت آن پاسخ را برای الف‌ها می‌فرستند تا مطمئن شوند الف‌ها می‌دانند پیامشان هم رسیده.

۵. اگر تا دو روز قبل از زمان مقرر دست کم یکی از این تاییدها ردوبدل شده باشد، دو طرف احتمال خطای نهایی را به شدت کاهش داده‌اند و می‌توانند بدون ارسال پیام بیشتر، بر اساس زمان از پیش تعیین شده حمله کنند.

در این روش، هرچقدر تعداد مراحل تایید بیشتر باشد، احتمال نرسیدن آن «آخرین تایید» پایین تر می‌رود اما هرگز به صفر نمی‌رسد.

۳. متن پیشنهادی برای پیام‌ها:

□ پیام اول از انسان به الف‌ها: «ما (فرمانده انسان‌ها) تصمیم گرفته‌ایم چهارشنبه هفته آینده با طلوع آفتاب از جبهه شمال شرقی حمله کنیم. اگر این پیام به شما رسید، با عبارت «پیغام شما دریافت شد؛ ما چهارشنبه هفته آینده از جبهه شمال غربی حمله می‌کنیم» پاسخ دهید. در غیر این صورت،

□ پاسخ اول از الف‌ها به انسان‌ها (تایید دریافت): «پیغام شما دریافت شد؛ ما چهارشنبه هفته آینده از جبهه شمال غربی حمله می‌کنیم.»

□ تایید نهایی از سوی انسان‌ها به الف‌ها: «پیغام تایید شما دریافت شد؛ هم‌زمان با شما در چهارشنبه هفته آینده با طلوع آفتاب از جبهه شمال شرقی حمله می‌کنیم.»

اگر هیچ‌یک از تاییدها تا دو روز قبل از حمله نرسید، هیچ‌کدام از ارتش‌ها دست به حمله تنها نمی‌زنند و منتظر موقعیت بعدی می‌شوند.

با ترکیب روش اول (تعیین زمان از پیش) و روش دوم (ارسال چند مرحله‌ای پیام با ACK)، می‌توان احتمال هماهنگی هم‌زمان را تا حد بسیار زیادی بهبود بخشید. اگر حداقل یک بار تایید تا دو روز قبل از حمله ردوبدل شود، هر دو فرمانده مطمئن می‌شوند طرف مقابل زمان حمله را می‌داند و حتماً هماهنگ خواهد بود. در نتیجه در لحظه طلوع آفتاب، سپاه انسان‌ها از شمال شرقی و سپاه الف‌ها از شمال غربی هم‌زمان هجوم برده و قلعه موردور با دفاع یکپارچه هم‌زمان توان مقاومت نخواهد داشت و سقوط خواهد کرد.

پاسخ سوال ۵

فرض کنید می‌خواهیم در یک سیستم توزیع شده که در آن ممکن است پیام‌ها تلف شوند و یا گره‌ها متوقف شوند عملیات همه پخش^۲ داشته باشیم. همه پخش باید این ویژگی را داشته باشد که یا تمام گره‌های فعال پیام آن را دریافت کنند و یا هیچ‌کدام آن را دریافت نکنند (یا دریافت کنند ولی تشخیص دهند که پیام به همه نرسیده و آن را دور بیندازند). قابل مشاهده است که اگر یک گره به تنهایی پیام خود را ارسال کند ممکن است پس از تحویل پیام به چند گره متوقف شود و گره‌های دیگر آن را دریافت نکنند. یکی از راه حل‌های مقابله با این مشکل این است که

تمام گره ها در همه پخشی شرکت کنند. به این ترتیب، هر گاه گره ای پیام همه پخشی از گره دیگری دریافت کند آن را برای گره های دیگر ارسال می کند. به این ترتیب حتی اگر گره اولیه متوقف شود، پیام او در نهایت به تمام گره های فعال می رسد. به این پروتکل همه پخشی قابل اتکای حریص^۳ می گویند. (آ) پیچیدگی این الگوریتم را محاسبه کنید. (ب) چگونه می توانید این الگوریتم را از نظر سربار پیام بهبود دهید؟

الف: محاسبه پیچیدگی الگوریتم در این الگوریتم ابتدا گره اولیه (که پیام را ایجاد کرده) پیام را به همه $N - 1$ گره دیگر (و می تواند خودش را هم بشمارد اما در عمل ارسال به خودش بی معنی است) ارسال می کند. بنابراین در مرحله اول تعداد $N - 1$ پیام فرستاده می شود. سپس هر یک از آن $N - 1$ گره های که پیام را دریافت کرده، برای بار دوم پیام را به همه $N - 1$ گره دیگر (شامل خود ارسال کننده اولیه یا سایر گره ها) می فرستند. عملاً اگر همه فرض کنیم پیام را دریافت کنند، در مرحله دوم تقریباً $(N - 1)^2 = (N - 1) \times (N - 1)$ ارسال داریم که نزدیک به N^2 است. به عبارت دیگر، در وضعیت عمومی که همه گره ها پیام را دریافت می کنند و دوباره منتشر می کنند، کل تعداد پیام های ارسالی در شبکه برابر

$$(N - 1) + (N - 1)^2 \approx N^2$$

خواهد بود. در نتیجه پیچیدگی این الگوریتم از نظر تعداد پیام های ارسالی برابر $O(N^2)$ است.

ب: بهبود الگوریتم از نظر سربار پیام در الگوریتم فوق ممکن است پیام یکسان بارها به یک گره برسد و آن گره نیز دوباره آن را به همه گره ها ارسال کند؛ در نتیجه تعداد پیام ها بسیار زیاد می شود. برای کاهش این سربار می توان از روش های زیر استفاده کرد:

۱. نگهداری شناسه پیام های منتشر شده: هر گره یک ساختار داده (مثلاً list یا set) از شناسه های پیام هایی که قبلاً منتشر کرده نگه می دارد. وقتی یک پیام جدید دریافت می کند، ابتدا شماره message ID را چک می کند. اگر قبلاً همین شناسه را در لیست داشته باشد، به این معناست که پیش تر پیام را منتشر کرده و نباید دوباره آن را بفرستد. در غیر این صورت شناسه را به لیست اضافه می کند و پیام را تنها یک بار برای گره هایی که هنوز پیام را دریافت نکرده اند ارسال می کند. این کار جلوی ارسال مکرر پیام های تکراری را می گیرد و تقریباً تضمین می کند هر گره حداکثر یک بار آن پیام را منتشر کند.

۲. استفاده از لیست دریافت کنندگان یا forwarding list: می توان در هر پیام یک forwarding list از شناسه گره هایی که تاکنون پیام را دریافت کرده اند گنجاند. وقتی گره ای پیام را دریافت می کند، اول این لیست را بررسی می کند و تنها به آن گره هایی ارسال می کند که شناسه شان در لیست نیست. سپس شناسه خودش را به لیست اضافه کرده و پیام را برای باقی گره های لازم می فرستد. به این ترتیب یک پیام فقط به زیرمجموعه های جدید ارسال می شود و جلوی ارسال های اضافی به گره هایی که پیش تر پیام را داشته اند گرفته می شود.

۳. استفاده از ساختار درخت پوششی (spanning tree): اگر بتوانیم یک spanning tree مستقل از وضعیت فعلی

شبکه بسازیم، هر گره پیام را تنها به فرزنداناش در درخت می فرستد. در نتیجه تعداد کل پیام های ارسالی تقریباً $N - 1$ خواهد بود (هر لبه درخت یک بار پیام را عبور می دهد). با وجود احتمال توقف موقت برخی گره ها، اگر درخت پوششی self-healing داشته باشیم یا چندین درخت پوششی متفاوت تعریف کنیم، می توانیم هم قابلیت اتکای بالا و هم سربار پایین را تا حد زیادی تضمین کنیم. در این حالت دیگر نیازی نیست هر گره پیام را به همه همسایه ها بفرستد، بلکه فقط به آن دسته از همسایه هایی ارسال می شود که زیرشاخه آن ها در درخت قرار دارد.

۴. گسترش با الگوریتم gossip-based broadcast: در این روش هر گره پیام را تنها به تعدادی از همسایه های تصادفی (به عنوان مثال $\log N$ همسایه) ارسال می کند. سپس هر همسایه با احتمال از پیش تعیین شده پیام را به دیگران مخابره می کند. مزیت این روش این است که تعداد ارسال ها به صورت احتمالی $O(N \log N)$ یا حتی $O(N)$ خواهد بود در حالی که با تکرارهای کافی احتمال آنکه پیام به تمام گره ها برسد بسیار بالا می رود. تنها نکته این است که تضمین قطعی برای رسیدن پیام به همه گره ها از بین می رود و به جای آن احتمال بسیار بالا (مثلاً $1 - \frac{1}{N}$) را به دست می آوریم. در محیط هایی که می توان سربار کمی را پذیرفت تا احتمال شکست را اساساً صفر کرد، این روش بسیار کارا است.

با اعمال هر یک از این بهینه سازی ها (نگهداری message ID، استفاده از forwarding list، درخت پوششی یا انتشار به سبک gossip-based broadcast) می توان سربار پیام در الگوریتم همه پخشی از $O(N^2)$ به تقریباً $O(N)$ یا $O(N \log N)$ کاهش داد و در عین حال تا حد زیادی تضمین کرد که پیام در نهایت به همه گره های فعال می رسد یا لااقل تشخیص داده می شود که پیام کامل به همه نرسیده است.

پاسخ سوال ۶

یک همه پخشی را ترتیب کلی FIFO می گوئیم اگر دو خاصیت زیر را داشته باشد:

۱. اگر یک گره دو پیام را یکی بعد از دیگری از طریق همه پخشی ارسال کند، آن گاه پیام ها به همان ترتیب همه پخشی به گره های دیگر تحویل می شوند.

۲. اگر پیام m1 در یک گره قبل از پیام m2 تحویل شود، آنگاه ترتیب تحویل این دو پیام در تمام گره های دیگر به همین صورت است.

شبه کدی بنویسید که یک همه پخشی ترتیب کلی FIFO را پیاده سازی کند. فرض کنید هر گره یک شناسه یکتا دارد و هر گره شناسه تمام گره های دیگر را می داند.

برای برآورده کردن این قیود، از ایده logical clock مستقل برای هر node استفاده می کنیم و یک شمارنده عددی می سازیم که با هر ارسال یک message مقدارش یکی افزایش یابد. این مقدار به عنوان sequence number به همراه خود message برای باقی node ها فرستاده می شود. وقتی هر message node ای را دریافت می کند، آن را در یک

buffer موقتی قرار می دهد تا زمانی که تمام message های با sequence number کوچکتر از همان sender تحویل داده شوند. تنها پس از آن اجازه می دهیم آن deliver message شود. با این روش:

□ اگر node X دو message به ترتیب با شماره های s و $s + 1$ ارسال کرده، همه node های دیگر ابتدا s را دریافت و تحویل می دهند و بعد $s + 1$ را، بنابراین ترتیب ارسال در هر نقطه ثابت می ماند (خاصیت ۱).

□ اگر node Y هر message $m1$ را با شماره s_1 زودتر از message $m2$ با شماره s_2 دریافت کند (یعنی $s_1 < s_2$)، همه node های دیگر هم به ترتیب منطقی sequence number همین ترتیب را خواهند داشت (خاصیت ۲).

در ادامه متغیرهای مورد نیاز و شبه کد کامل تر را می بینید.

متغیرها در هر node:

□ $\text{clock} \leftarrow *$

□ $\text{buffer} \leftarrow \{\}$

□ $\text{lastDelivered}[\text{pid}] \leftarrow *$ برای همه pid

□ $\text{allNodes} \leftarrow$ لیست شناسه تمام node ها

شبه کد کامل تر:

```

1 def FIFO_Broadcast(msg):
2     clock ← clock + 1
3     seq ← clock
4     for each q in allNodes:
5         send(msg, seq, self_id) to q
6
7 def on_receive(msg, seq, sender_id):
8     buffer.append((msg, seq, sender_id))
9     try_deliver()
10
11 def try_deliver():
12     repeat:

```

```

13     delivered ← false
14     for each (m, s, p) in buffer:
15         if s == lastDelivered[p] + 1:
16             deliver(m, p)
17             lastDelivered[p] ← s
18             buffer.remove((m, s, p))
19             delivered ← true
20     until not delivered
21
22 def deliver(m, p):
23     print("Delivered_message_from", p, ":", m)

```

این الگوریتم تضمین می کند که:

□ هر node با هر فراخوانی $sequence\ number_{FIFO_{broadcast}}$ منحصر به فرد تولید کرده و ارسال می کند.

□ در تابع $message_{try_deliver}$ از buffer جدا شده و deliver می شود که همه message های قبلی آن sender دریافت و تحویل شده باشند.

مراجع